



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树



二叉树遍历递归算法



■ 优点

- 代码简洁易懂
- 逻辑清晰
 - 直接对应遍历的数学定义
 - 易于理解和验证正确性
- 实现快速
 - 几行代码即可完成
 - 适合快速原型开发
- 自动栈管理
 - 系统自动处理函数调用栈
 - 无需手动管理状态

■ 缺点

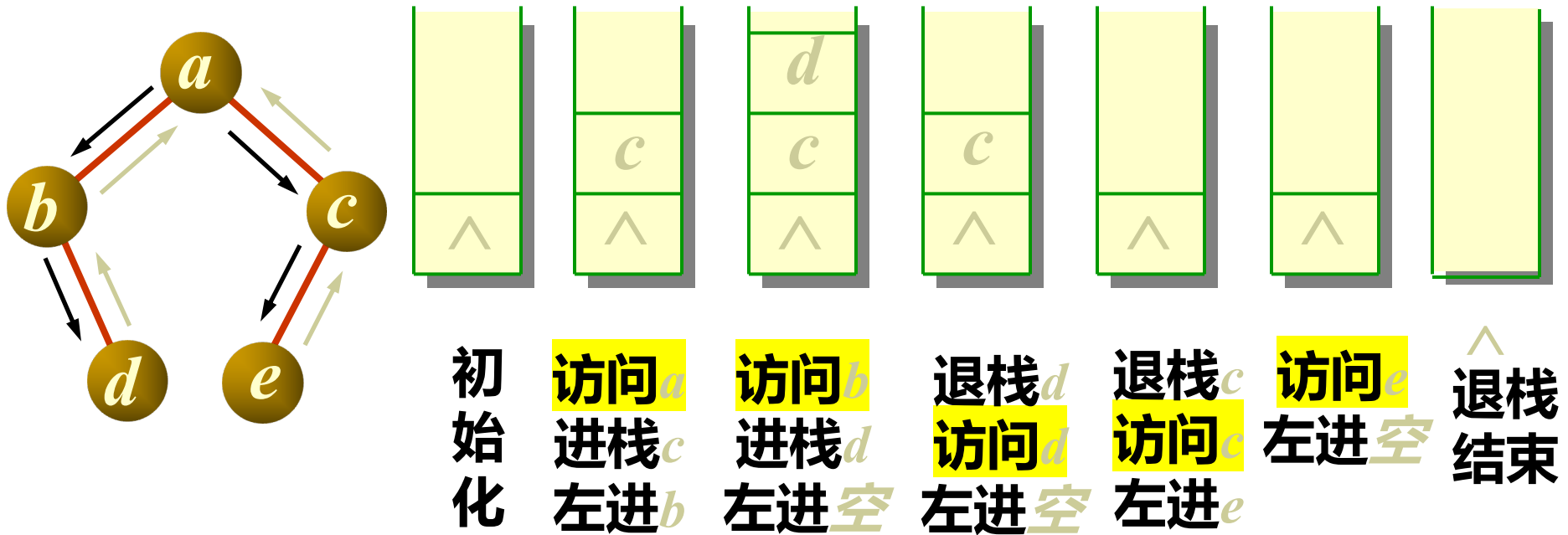
- 栈溢出风险
 - 树深度过大时可能导致栈溢出
 - 受系统栈大小限制
- 性能开销大
 - 函数调用开销较大
 - 递归深度大时效率较低
- 调试困难
 - 递归调用栈较难跟踪
 - 不易观察中间状态



5.4.3 二叉树遍历的非递归算法



1. 利用栈的前序遍历非递归算法



利用栈记录遍历时的回退路径



1. 利用栈的前序遍历非递归算法



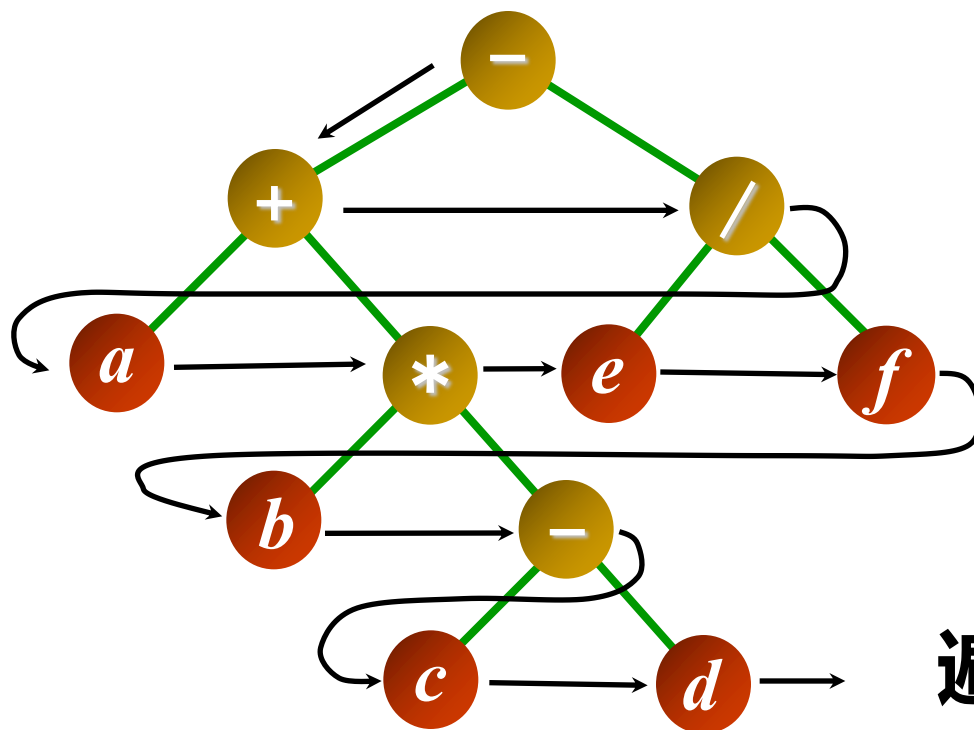
```
template <class T>
void BinaryTree<T>::
PreOrder (void (*visit) (BinTreeNode<T> *t) ) {
    stack<BinTreeNode<T>*> S;
    BinTreeNode<T> *p = t;
    S.Push (NULL);
    while (p != NULL) {
        visit(p);                //访问结点
        if (p->rightChild != NULL) S.Push (p->rightChild);
                                //预留右指针在栈中
        if (p->leftChild != NULL) p = p->leftChild;
                                //进左子树
        else S.Pop(p);           //左子树为空，出栈
    }
};
```



2. 层次序遍历二叉树的算法



- **层次序遍历**二叉树就是从根结点开始，按层次逐层遍历，如图：



遍历顺序



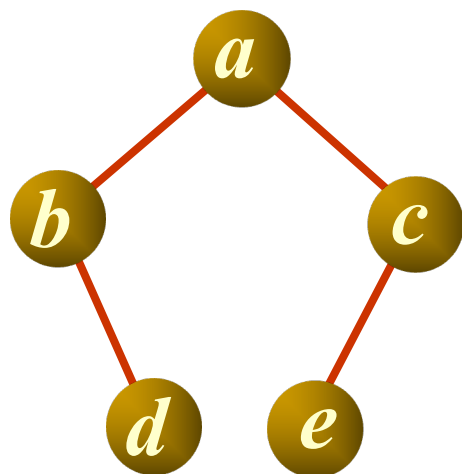
2. 层次序遍历二叉树的算法



- 这种遍历需要使用一个**先进先出的队列**，在处理上一层时，将其下一层的结点直接进到队列（的队尾），然后再访问已在队列队头的结点。
- 在上一层结点遍历完后，下一层结点正好处于队列的队头，可以继续访问它们。
- 算法是非递归的。



2. 层次序遍历二叉树的算法



Q

<i>a</i>				
----------	--	--	--	--

*a*进队

Q

	<i>b</i>	<i>c</i>		
--	----------	----------	--	--

*a*出队, 访问*a*
*b*进队, *c*进队

Q

		<i>c</i>	<i>d</i>	
--	--	----------	----------	--

*b*出队, 访问*b*
*d*进队

Q

			<i>d</i>	<i>e</i>
--	--	--	----------	----------

*c*出队, 访问*c*
*e*进队

Q

				<i>e</i>
--	--	--	--	----------

*d*出队, 访问*d*

Q

--	--	--	--	--

*e*出队, 访问*e*



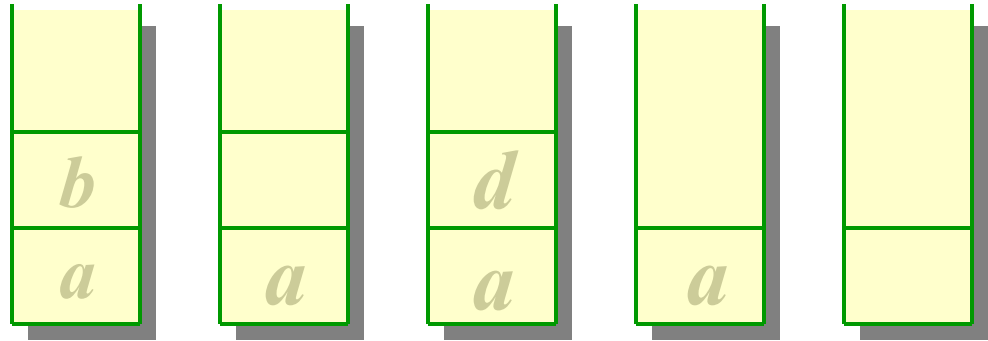
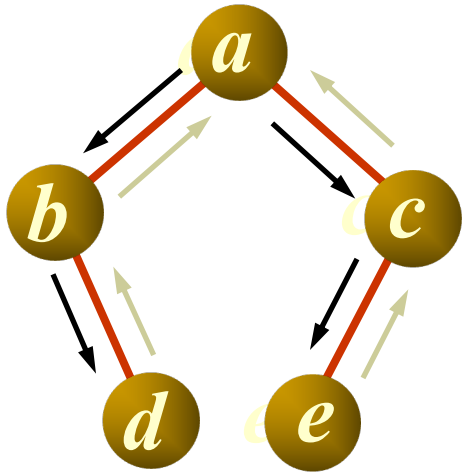
2. 层次序遍历二叉树的算法



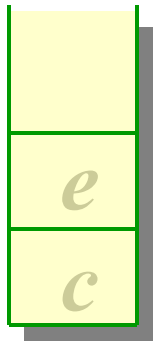
```
template <class T>
void BinaryTree<T>::
levelOrder (void (*visit) (BinTreeNode<T> *t)) {
    if (t == NULL) return;
    Queue<BinTreeNode<T> * > Q;
    BinTreeNode<T> *p = t;
    Q.Enqueue (p);
    while (!Q.IsEmpty ()) {
        Q.DeQueue (p); visit (p);
        if (p->leftChild != NULL) Q.Enqueue (p->leftChild);
        if (p->rightChild != NULL) Q.Enqueue (p->rightChild);
    }
};
```



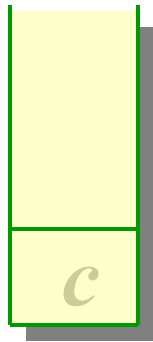
3. 利用栈的中序遍历非递归算法



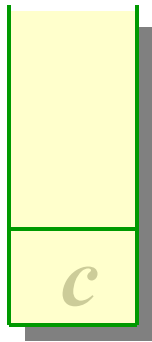
b左空 b退栈 d进栈 d退栈 a退栈
访问b d左空 访问d 访问a



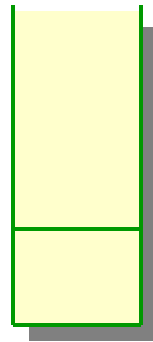
e左空



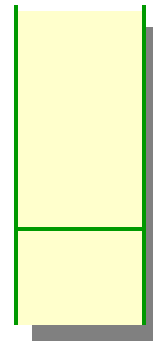
e退栈
访问e



c右空



c退栈
访问c



栈空
结束



3.利用栈的中序遍历非递归算法



```
template <class T>
void BinaryTree<T>::
InOrder (void (*visit) (BinTreeNode<T> *t)) {
    stack<BinTreeNode<T>*> S;
    BinTreeNode<T> *p = t;
    do {
        while (p != NULL) {           //遍历指针向左下移动
            S.Push (p);               //该子树沿途结点进栈
            p = p->leftChild;
        }
        if (!S.IsEmpty()) {           //栈不空时退栈
            S.Pop (p); visit (p);     //退栈, 访问根结点
            p = p->rightChild;        //遍历指针进到右子女
        }
    } while (p != NULL || !S.IsEmpty ());
};
```



4. 利用栈的后序遍历非递归算法



- 在后序遍历过程中所用栈的结点定义

```
template <class T>
```

```
struct stkNode {
```

```
    BinTreeNode<T> *ptr;    //树结点指针
```

```
    enum tag {L, R};        //退栈标记
```

```
    stkNode (BinTreeNode<T> *N = NULL) :
```

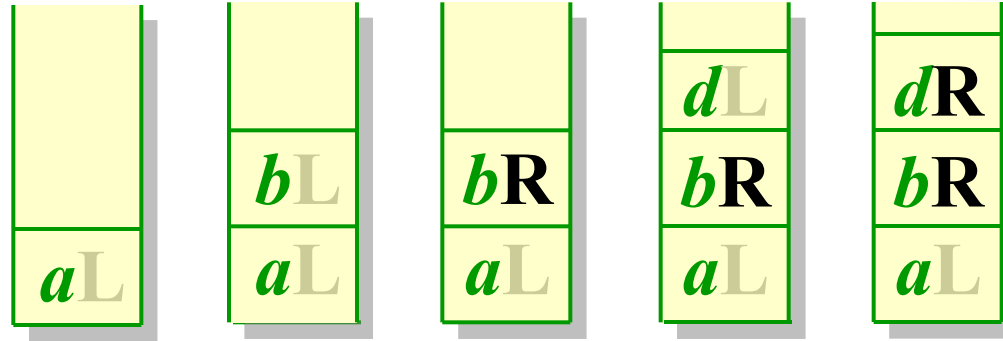
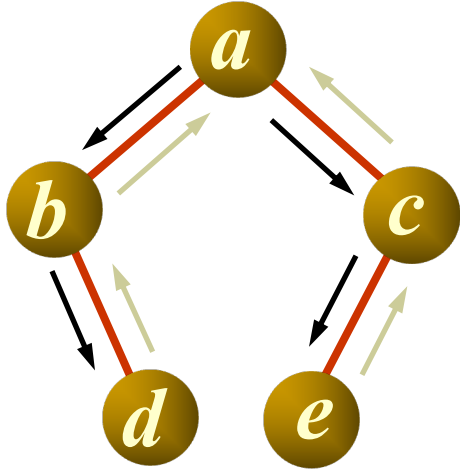
```
        ptr(N), tag(L) { }    //构造函数
```

```
};
```

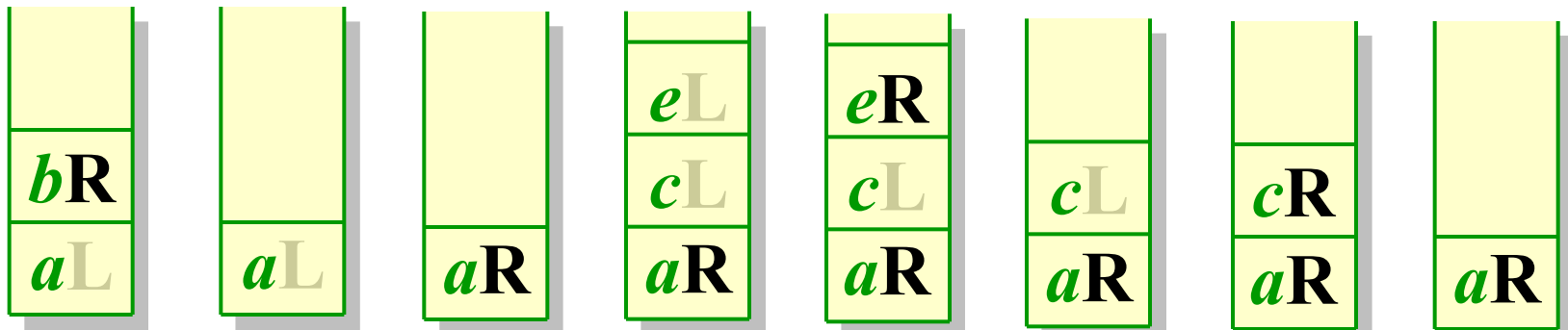
- tag = L, 表示从左子树退回还要遍历右子树;**
tag = R, 表示从右子树退回要访问根结点。



4. 利用栈的后序遍历非递归算法



d退栈
访问d



b退栈
访问b

e退栈
访问e

c退栈
访问c

a退栈
访问a



4. 利用栈的后序遍历非递归算法



```
template <class T>
void BinaryTree<T>::
PostOrder (void (*visit) (BinTreeNode<T> *t) {
    Stack<stkNode<T>> S;  stkNode<T> w;
    BinTreeNode<T> * p = t;  //p是遍历指针
    do {
        while (p != NULL) {
            w.ptr = p; w.tag = L; S.Push (w);
            p = p->leftChild;
        }
        int continue1 = 1;  //继续循环标记, 用于R
```



4. 利用栈的后序遍历非递归算法



```
while (continue1 && !S.IsEmpty ()) {  
    S.Pop (w); p = w.ptr;  
    switch (w.tag) {           //判断栈顶的tag标记  
        case L: w.tag = R; S.Push (w);  
                continue1 = 0;  
                p = p->rightChild; break;  
        case R: visit (p); break;  
    }  
}  
} while (!S.IsEmpty ());      //继续遍历其他结点  
cout << endl;  
};
```



二叉树遍历非递归算法



■ 优点

- 无栈溢出风险
- 性能更优
 - 避免函数调用开销
 - 适合深度大的树
- 可控性更强
 - 可以手动控制遍历过程
 - 易于暂停、恢复遍历
- 内存使用明确
 - 显式栈大小可预测
 - 便于内存管理

■ 缺点

- 代码复杂
 - 实现相对繁琐
 - 容易出错
- 可读性差
 - 业务逻辑被栈操作掩盖
 - 初学者难以理解
- 维护成本高
 - 修改遍历逻辑较困难
 - 需要手动管理状态



■ 使用队列或栈非递归实现二叉树

层序	使用队列
前序	右孩进栈；左空退栈
中序	左到底；左空退栈；右孩进栈
后序	左到底；看tag: $L \rightarrow R$; R退栈

1. 关于前序遍历的非递归实现，以下说法正确的是：

- A) 需要使用队列来存储节点
- B) 右子树要先于左子树入栈
- C) 左子树要先于右子树入栈
- D) 不需要使用栈结构



2. 中序遍历的非递归实现中，内层**while**循环的作用是：

- A) 遍历所有右子树节点
- B) 将当前节点及其所有左子节点入栈
- C) 输出根节点的值
- D) 检查栈是否为空

3. 层次遍历中，队列出队的顺序就是树的 _____ 顺序。

4. 对于二叉树 **[A,B,C]** (**B**是左孩子，**C**是右孩子)，非递归前序遍历的栈操作序列是：

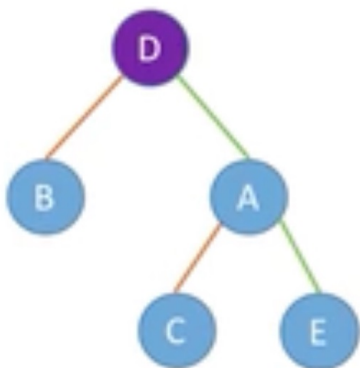
入栈 _____ → 出栈 _____ → 入栈 _____ → 入栈 _____



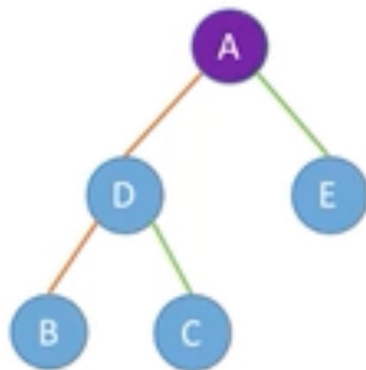
5.4.4 二叉树的计数



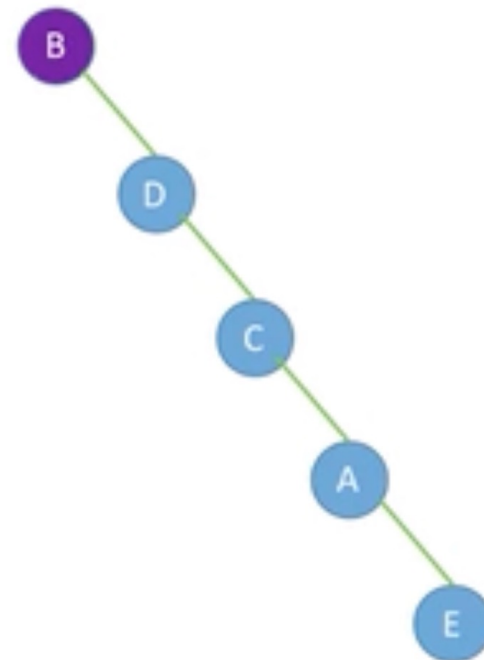
中序遍历：中序遍历左子树、根结点、中序遍历右子树



中序遍历序列：BDCAE



中序遍历序列：BDCAE

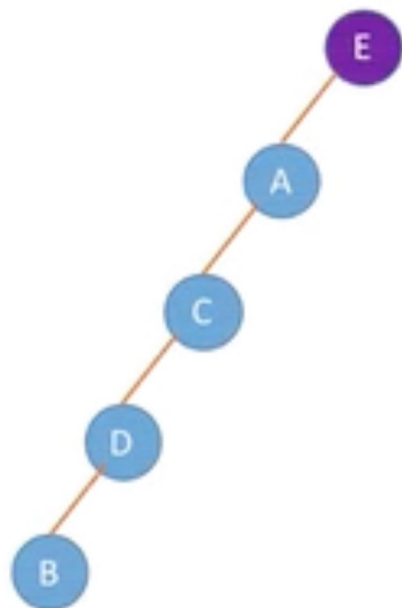


中序遍历序列：BDCAE

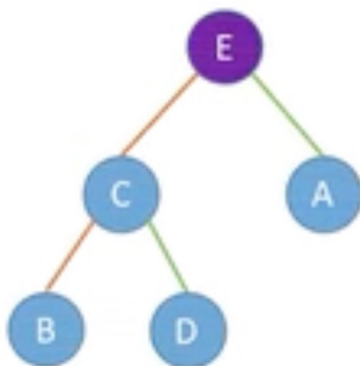
一个中序遍历序列可能对应多种二叉树形态



后序遍历：前序遍历左子树、前序遍历右子树、根结点

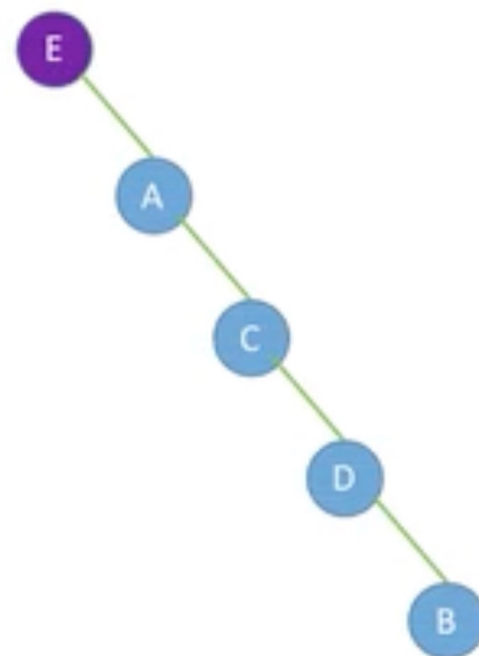


后序遍历序列：BDCAE



后序遍历序列：BDCAE

结论：一个后序遍历序列
可能对应多种二叉树形态



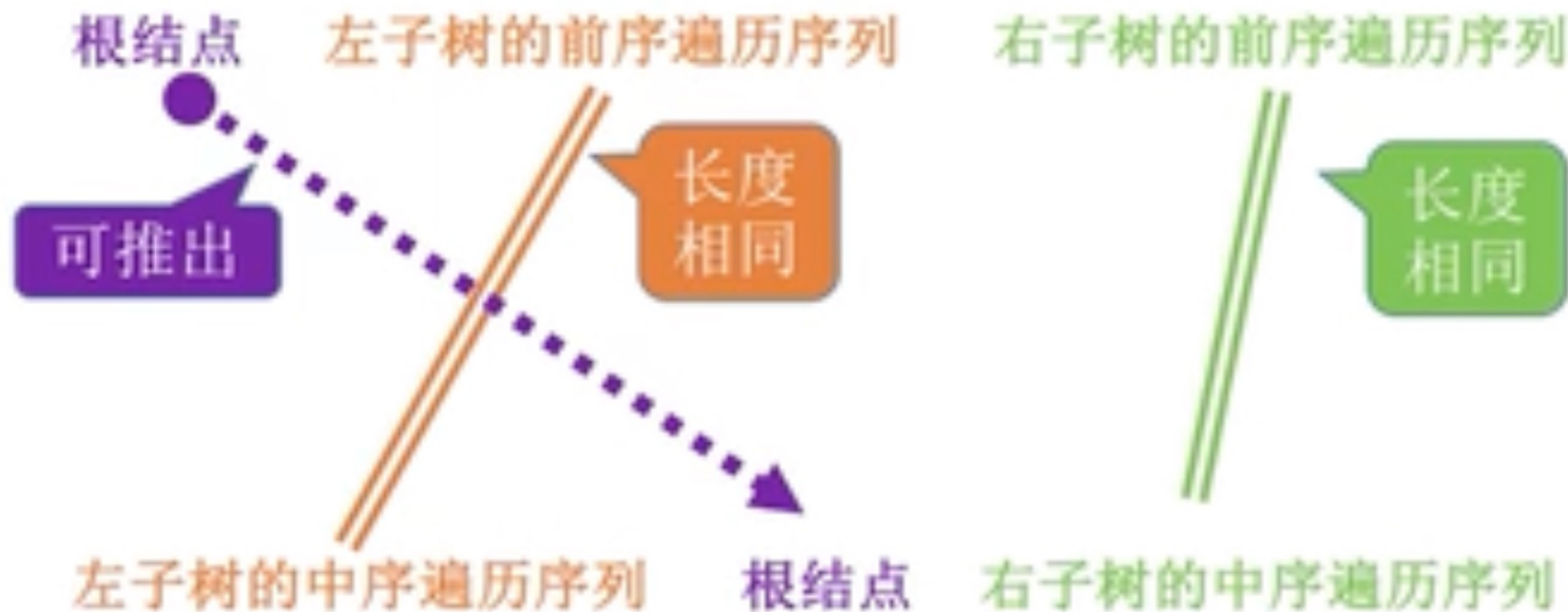
后序遍历序列：BDCAE



- 由给定的前序序列和中序序列能够唯一地确定一棵二叉树
- 例如：假定一棵二叉树的前序序列为
AHBFDECKG， 中序序列为HBDFAEKCG



前序+中序





后序+中序



左子树的后序遍历序列

长度
相同

左子树的中序遍历序列

右子树的后序遍历序列

长度
相同

根结点

右子树的中序遍历序列

根结点

可推出



层序+中序

